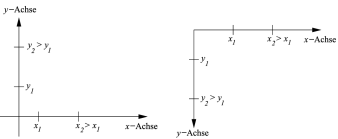


1. GRUNDLAGEN

Das folgende Bild vergleicht ein Koordinatensystem in der Mathematik mit einem Koordinatensystem auf dem Computerbildschirm. Die *y*-Achse zeigt auf dem Bildschirm nach unten.



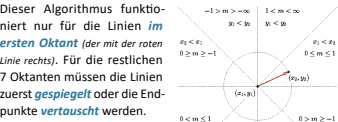
Punkt: wird mit einer *x*- und einer *y*-Koordinate beschrieben: $P = (p_x, p_y)$
Vektor: Verbindung von zwei Punkten im Koordinatensystem. $\vec{p} = (p_x, p_y)^T$ ist der Vektor, der von 0 zu *P* führt.
Polygon: Folge von Punkten, verbunden mit Linien, mit selbstem Start- & Endpunkt. **Konvex:** Von jeder beliebigen Ecke des Polygons kann eine gerade Linie zu allen anderen Ecken gezogen werden. **Konkav:** Bei mindestens einer Ecke geht das nicht.

2. LINIEN

Aus einem *Anfangs- und einem Endpunkt* $P_1 = (x_1, y_1)$ und $P_2 = (x_2, y_2)$ einer Linie *l* sind die **rasterisierten Pixel** zu berechnen.

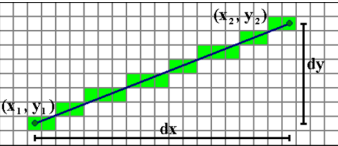
2.1. BRESENHAM-ALGORITHMUS

Der Algorithmus funktioniert so, dass man in der **«schnellen» Richtung**, also die Richtung, in die die Linie eher geht, **immer einen Pixel färbt**, und in die andere Richtung nur falls nötig.



- 1. Falls $x_2 < x_1$:
 x_1 mit x_2 tauschen
- 2. Falls $(y_2 - y_1) < 0$:
Punkte an *x*-Achse spiegeln
- 3. Falls $(y_2 - y_1) > (x_2 - x_1)$: Steigung ist grösser als 45°, Linie muss an 45°-Achse gespiegelt werden.

```
zeichneLinie(startX, startY, endX, endY) // Bresenham Pseudocode
deltax = endX - startX; deltay = endY - startY
steigung = deltay / deltax
x = startX; y = startY; fehler = 0
while x <= endX
  setzePixel(x, y); x = x + 1
  fehler = fehler + steigung
  if fehler > 0.5
    y = y + 1; fehler = fehler - 1
```



2.2. ANTI-ALIASING

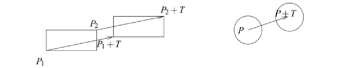
Um keine **«Ecken»** in der Linie zu haben, können die Pixel **prozentual** zum Anteil der Linie im Pixel **gefärbt** werden. **Beispiel:** 60% Überlappung mit Ideallinie → Pixel hat 60% Grauwert.

3. 2D-TRANSFORMATIONEN

Mithilfe von Transformationen ist es möglich, die **Position**, die **Orientierung**, die **Form** und die **Grösse** von grafischen Objekten zu **manipulieren**. Transformationen eines Objekts werden durch die **Operationen** auf den **Definitionspunkten** realisiert. Durch 2 Punkte definierte Rechtecke müssen zunächst zu **Polygonen** gemacht werden (Rechteck wird in zwei Dreiecke aufgeteilt).

3.1. TRANSLATION

Eine **gradlinige Verschiebung** um einen Translationsvektor *T* ist definiert durch $T := (t_x, t_y)$, d.h. wird zu jedem Punkt P_i die Translation dazu gezählt: $P' = P + T$.



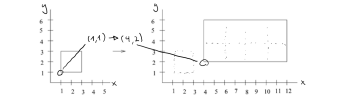
3.2. SKALIERUNG

Durch Skalierung wird jeder Punkt anhand eines **Fixpunktes** *s* vergrössert bzw. verkleinert. Im einfachsten Fall liegt der Fixpunkt im Ursprung (0,0): $(x', y') := (s_x \cdot x, s_y \cdot y)$

Wenn $s_x = s_y$ handelt es sich um eine **uniforme Skalierung** (Proportionen bleiben erhalten), bei $s_x \neq s_y$ um eine **Verzerrung**. Damit das Objekt zweidimensional bleibt, muss $s_x, s_y \neq 0$ sein.

3.2.1. Beispiel

Folgendes Quadrat wird mit $s_x = 4, s_y = 2$ skaliert (Verzerrung):



3.2.2. Skalierung mit Fixpunkt

Bei Skalierung mit einem Fixpunkt (Z_x, Z_y) für den Punkt *P* müssen die folgenden drei Schritte abgearbeitet werden:

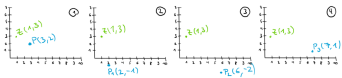
- 1. **Translation** um $(-Z_x, -Z_y)$. Verschiebt *P* zum Nullpunkt, dies ist nun P_1 (Damit ist eine Skalierung ohne Verschiebung möglich)
- 2. **Skalierung** mit (s_x, s_y) liefert P_2 .
- 3. **Translation** um (Z_x, Z_y) liefert $P_3 = P'$ (Zurück zum Startpunkt).

Zusammengefasst ergibt das diese Formel für die Berechnung:

$(x', y') := ((x - Z_x) \cdot s_x + Z_x, (y - Z_y) \cdot s_y + Z_y)$

Beispiel: Skalierung mit $s_x = 3, s_y = 2, Z = (1, 3)$

- 1. **Ausgangslage:** *P* ist bei $x = 3$ und $y = 2$.
- 2. **Translation** um $(-Z_x = -1, -Z_y = -3)$:
 $P(3, 2)$ zu $(-1, -3) = P_1(2, -1)$
- 3. **Skalierung** mit $(s_x = 3, s_y = 2)$:
 $P_1(2, -1) \cdot (3, 2) = P_2(6, -2)$
- 4. **Translation** um (Z_x, Z_y) :
 $P_2(6, -2)$ zu $(1, 3) = P_3(7, 1) = P'$.



Oder eingesetzt in die Formel:

$(x', y') := ((3 - 1) \cdot 3 + 1, (2 - 3) \cdot 2 + 3) = (7, 1)$

3.3. ROTATION

Die Drehung des Objekts anhand eines **Fixpunktes** um einen Drehwinkel β . Wenn der Fixpunkt im **Ursprung** liegt, gilt:

$x' = L \cdot \cos(\alpha + \beta)$
 $y' = L \cdot \sin(\alpha + \beta)$
...
 $x' = x \cdot \cos(\beta) - y \cdot \sin(\beta)$
 $y' = x \cdot \sin(\beta) + y \cdot \cos(\beta)$

Positive Werte für β bewirken eine **Rotation gegen** (Koordinatensystem) bzw. **mit dem Uhrzeigersinn** (Bildschirm).

Siehe Bild in Kapitel 12.3. «Koordinatensystemwechsel» für cos / sin. Werte.

3.3.1. Beliebiges Rotationszentrum

Bei Wahl eines beliebigen Rotationszentrums (R_x, R_y) folgt für den Punkt *P* (Analog zu Kapitel 3.2.2. «Skalierung mit Fixpunkt»):

- 1. **Translation** um $(-R_x, -R_y)$ liefert P_1
- 2. **Rotation** im Ursprung um Winkel β liefert P_2
- 3. **Translation** um (R_x, R_y) liefert $P_3 = P'$

3.4. MATRIXDARSTELLUNG

Kann mehrere Transformationen zu einer zusammensetzen, um Rundungsfehler zu vermindern. So sieht eine **Skalierung** in Matrixschreibweise aus:

$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x \cdot s_x \\ y \cdot s_y \end{pmatrix}$

Und so sieht eine **Rotation** in Matrixschreibweise aus:

$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos(\beta) & -\sin(\beta) \\ \sin(\beta) & \cos(\beta) \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x \cdot \cos(\beta) - y \cdot \sin(\beta) \\ x \cdot \sin(\beta) + y \cdot \cos(\beta) \end{pmatrix}$

3.4.1. Homogene Koordinaten

Für eine **Translation** mit **Matrizen** benötigen wir eine dritte Koordinate. Ein Punkt $P = (x, y)$ hat mit $w \neq 0$ (*w* ist meistens 1, falls nicht, können *x, y, w* durch *w* geteilt werden) folgende homogenen Koordinaten:

$\begin{pmatrix} x \\ y \\ w \end{pmatrix}$

Richtungsvektor

Der Richtungsvektor $\vec{r}$, der vom Ursprung zum Punkt $R = (x, y)$ führt, hat die **homogenen Koordinaten** $(x \ y \ 0)^T$

3.5. ALLGEMEINE TRANSFORMATIONEN

Translation

$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$

Skalierung

$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x \cdot s_x \\ y \cdot s_y \\ 1 \end{pmatrix}$

Rotation

$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos(\beta) & -\sin(\beta) & 0 \\ \sin(\beta) & \cos(\beta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x \cdot \cos(\beta) - y \cdot \sin(\beta) \\ x \cdot \sin(\beta) + y \cdot \cos(\beta) \\ 1 \end{pmatrix}$

Beispiel: Zusammengesetzte Transformation: 60° Rotation um $(-3, 5)$. (A: Translation Nullpunkt, B: Rotation, C: Rücktranslation)

$A = \begin{pmatrix} 1 & 0 & -3 \\ 0 & 1 & -5 \\ 0 & 0 & 1 \end{pmatrix}, B = \begin{pmatrix} 0.5 & -0.866 & 0 \\ 0.866 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}, C = \begin{pmatrix} 1 & 0 & 3 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{pmatrix}$

$D = C \cdot B \cdot A = \begin{pmatrix} 0.5 & -0.866 & 5.83 \\ 0.866 & 0.5 & -0.098 \\ 0 & 0 & 1 \end{pmatrix}$

Ist die Transformation kommutativ?

1. ↓, 2. →	Translation	Skalierung	Rotation
Translation	✓	✗	✗
Skalierung	✗	✓	✗
Rotation	✗	✗	✓

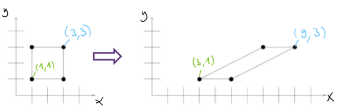
Skal. und Rot. sind nur kommutativ, wenn Skalierung in *x* und *y*-Richtung gleich ist.

Weitere Transformationen, die sich durch Matrizen darstellen lassen, sind: **Spiegelung** an einer beliebigen **Geraden**, **Spiegelung** an einem **Punkt** und **Scherung**.

Bei einer Scherung in *x*-Richtung bleiben die *y*-Werte konstant, und die *x*-Werte werden proportional zu den *y*-Werten horizontal verschoben, also $x' = x + Sch_x \cdot y$. Transformationsmatrix:

Scherung in x-Richtung
 $\begin{pmatrix} 1 & Sch_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
Scherung in y-Richtung
 $\begin{pmatrix} 1 & 0 & 0 \\ Sch_y & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Beispiel: Finde die Transformationsmatrix



- 1. Ein Punkt in Gleichung einsetzen, bspw. Punkt links unten:
 $\begin{pmatrix} 1 & Sch_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}$
- 2. Dies ergibt folgende Gleichung, um Sch_x herauszufinden:
 $1 \cdot 1 + Sch_x \cdot 1 + 0 \cdot 1 = 3$
 $1 + Sch_x = 3 \quad | -1$
 $Sch_x = 2$
- 3. Kontrolle mit anderem Punkt (hier Punkt rechts oben):
 $\begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 3 + 2 \cdot 3 + 0 \cdot 1 \\ 0 \cdot 3 + 1 \cdot 3 + 0 \cdot 1 \\ 0 \cdot 3 + 0 \cdot 3 + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 9 \\ 3 \\ 1 \end{pmatrix} \checkmark$

$Sch_x = 2$, da *y* 2-mal grösser ist als *x*. Kann auch aus direkt aus Bild herausgelesen werden (auf 2x verschiebt sich Punkt um 4y).

4. CLIPPING

Clipping ist das **Abschneiden von Objekten** am Rand eines gewünschten Bildschirmsschnittes oder Fensters. Dafür werden für eine Menge von Linien jeweils **neue Anfangs- und Endpunkte** bestimmt, welche innerhalb des Clip-Fensters liegen.

4.1. COHEN-SUTHERLAND ALGORITHMUS

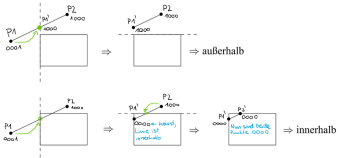
Teile Ebene anhand des Clip-Fensters in **9 Bereiche** ein, beschrieben durch **4-Bit-Bereichscode**. 0000 ist der sichtbare Bereich.

Codierung:

- XXX1: links vom Fenster (Bit 0) — X1XX: unter dem Fenster
- XX1X: rechts vom Fenster — XXXX: über dem Fenster

1001	1000	1010
0001	0000	0010
0101	0100	0110

Die Anfangs- und Endpunkte von Linien werden durch die Funktion **Code(P)** mit dem Code des Bereichs versehen, in welchem der Punkt liegt. Wenn **1 Bit** von Anfang- und Endpunkt **gleich** ist, ist die Linie **komplett ausserhalb** des Clipping-Fensters. Es wird im (Gegen-)Uhrzeigersinn jeder Rand des Fensters geclippt.



4.1.1. Implementierung

byte region_code(Point P): Setzt den Region Code pro Anfangs- und Endpunkt wie oben beschrieben.
void set_clip_window(Point P, Point delta): Setzt die Variablen xmin, ymin auf den Ursprung *P* des Clip-Fensters und xmax, ymax auf die Breite/Höhe δ .
boolean cohenSutherland(Point p1, Point p2, Point Q1, Point Q2): Liefert true, falls die Gerade p1 – p2 sichtbar ist. Der sichtbare Teil Q1 – Q2 ist in Out-Parametern Q1 & Q2.

5. 2D-FÜLLEN

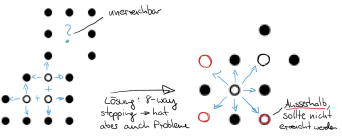
Es gibt **zwei Ansätze** zum Füllen eines Objekts mit Farbe oder Muster: Universell und Scan-Line.

5.1. UNIVERSELLE VERFAHREN

Stützen sich auf die **Nachbarschaft** eines Pixels. Sind **sehr einfach rekursiv** umzusetzen. Ein Nachteil ist der **hohe Speicherbedarf**, der durch die Rekursion benötigt wird. Zum **Starten** wird eine Begrenzung und ein Punkt innerhalb der Form benötigt.

- **4-way-stepping:** Überprüft in 4 Richtungen, ob der nächste Punkt ebenfalls innerhalb der Form ist.
- **8-way-stepping:** Überprüft in 8 Richtungen, ob der nächste Punkt ebenfalls innerhalb der Form ist.

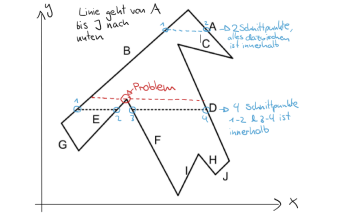
Beide Varianten haben ihre Probleme mit der Erreichbarkeit der Pixel. 4-way-stepping reicht aber meist aus.



5.2. SCAN-LINE-VERFAHREN

Bewegt eine **waagerechte Scan-Linie** schrittweise von oben nach unten über das Polygon, und berechnet die Schnittpunkte der Scan-Linie mit dem Polygon.

- 1. **Sortiert** alle Kanten nach ihrem grössten *y*-Wert (hier Liste A-J).
- 2. **Bewegt** die Scan-Linie vom grössten bis zum kleinsten *y*-Wert.
- 3. Für **jede Position** der Scan-Linie werden:
 - die **Liste der aktiven Polygonkanten** ermittelt (Kanten, welche die Scanline berührt)
 - die **Schnittpunkte berechnet** und nach *x*-Werten **sortiert**
 - jene **Scan-Line-Segmente**, die im **Inneren** des Polygons liegen, **angezeigt**



Problematik: Wenn die Scan-Linie an einem Schnittpunkt zweier Kanten (*K₁*, *K₂*) ankommt, muss direkt ein neues Segment begonnen werden.

6. 2D GRAFIKEN IM WEB (SVG)

SVG (Scalable Vector Graphics) ist ein offener Standard eines Vektor-grafikformates auf XML-Basis.

- + Kann nach Text durchsucht werden
- + Erlaubt Textgestaltung
- + Kann Objekte zeitlich koordiniert bewegen
- + Enthält Event Handling
- + verfügt über photoshop-artige Filter-Effekte
- + ist sowohl unkomprimiert als auch komprimiert einsetzbar
- verlangt Rechenleistung auf der Clientseite
- erfordert zum Abspielen ein Plugin (heute nicht mehr)

Beispiel Kreis

```
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg width="200" height="200" >
  <circle cx="100" cy="100" r="90" />
</svg>
```

6.1. GRUNDELEMENTE

```
<svg version="1.1" xmlns="http://www.w3.org/2000/svg"
xmlns:xlink="http://www.w3.org/1999/xlink">
<title>Gerade, Rechteck, Kreis, Ellipse, Polygon, Text</title>
<rect x="50" y="64" fill="#A6A60F" width="332" height="127"/>
<circle fill="#8F8CBA" stroke="#8B9654" stroke-width="2" cx="52"
cy="125" r="69"/>
<polygon fill="#27696F" stroke="#14A649" stroke-width="2"
points="231,223 305,132 258,36 307,22 366,65 409,133 353,223"/>
<ellipse fill="#F2522F" stroke="#090825" stroke-width="2"
cx="187" cy="117" rx="25" ry="95"/>
<line fill="none" stroke="#A6A60F" stroke-width="8" x1="25"
y1="247" x2="433" y2="267"/>
<text style="font-family:JetBrains Mono; font-size:30px;
font-weight:bold" x="25" y="255">Veränderung und Füllung</text></svg>
```



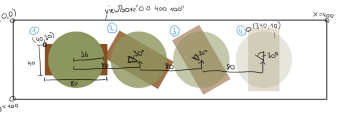
6.2. GRUPPIERUNG VON ELEMENTEN

Mit dem **<g>-Element** können mehrere Objekte zusammengefasst werden. Werden über das `xlink:href`-Element referenziert (wie *id* in HTML) und mit transform transformiert.

Achtung: Transformationen beziehen sich auf das Koordinatensystem des Objektes.
transform="translate (200,0) rotate(45)" bedeutet zuerst eine Drehung im Uhrzeigersinn um 45° und dann eine Verschiebung um 200 Pixel längs der Diagonalen nach rechts unten.

Beispiel

```
<svg width="400" height="100" viewBox="0 0 400 100" ... >
<defs>
  <g id="vorlage">
    <rect x="40" y="20" width="80" height="40" fill="#A6A60F" />
    <circle cx="0" cy="0" r="36" fill="#8B9654" />
  </g>
  <use xlink:href="#vorlage" transform="translate(80 50)" />
  <use xlink:href="#vorlage"
transform="translate(160 50) rotate(30)" fill-opacity="0.8"/>
  <use xlink:href="#vorlage"
transform="translate(240 50) rotate(60)" fill-opacity="0.5"/>
  <use xlink:href="#vorlage"
transform="matrix(0.8 1.0 -1.0 0.8 320 50)" fill-opacity="0.2"/>
</svg>
```



Der *x* und *y*-Wert des rect definiert, wo die obere linke Ecke liegt. Mit *x*="40", *y*="20" und der ersten Translation von (80 50) führt das dazu, dass die obere Ecke zuerst bei *x*="40" und *y*="30" liegt (Ohne die Minuswerte wäre die Ecke im Kreismittelpunkt).

Matrix-Transformation

Die Matrix-Transformation in SVG hat 6 Parameter.
matrix(a b c d e f) = $\begin{pmatrix} a & b & c \\ b & d & f \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.0 - 1.0 320 \\ 1.0 0.0 50 \\ 0 & 0 & 1 \end{pmatrix}$

Nun kann man die einzelnen Ecken mit einer Matrix Transformation umrechnen und erhält so die neuen Werte (siehe Kapitel 3.5. «Allgemeine Transformationen»).

Im Beispiel wird der Punkt (40,30) wie folgt umgewandelt:

$\begin{pmatrix} x_1 \\ y_1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 320 \\ 1 & 0 & 50 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} -40 \\ -20 \\ 1 \end{pmatrix} = \begin{pmatrix} -20 + 320 \\ -40 + 50 \\ 1 \end{pmatrix} = \begin{pmatrix} 340 \\ 10 \\ 1 \end{pmatrix}$

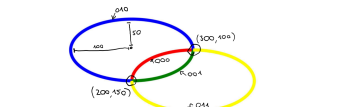
6.3. PFADE

path-Elemente zeichnen Kurven. Der Pfad ist im Attribut *d* definiert: <path d="...". Im Attribut können sich unter anderem folgende Kommandos befinden:

- **M:** moveto (von einem Punkt zum nächsten bewegen)
- **A:** arcs (Erstellt einen Kreisbogen)
- **C:** curveto (Erstellt eine Kurve)
- **S:** smooth curveto (Erstellt eine glatte Kurve)
- **L:** lineto (Zeichnet gerade Linie zu *x* und *y* Koordinaten)

Kreisbögen

```
<!-- Argumente für SVG Arcs -->
d="A rx ry x-axis-rotation large-arc-flag sweep-flag x y"
<path fill="none" stroke="red" da">
  A 300 100 <!-- starting point at (300,100) -->
  A 100 50 <!-- make an arc, x=Radius = 100, y=Radius = 50 -->
  0 <!-- x-axis-rotation in degrees -->
  0 <!-- large-arc-flag: 0 short way, 1 long way -->
  0 <!-- sweep-flag: 0 counter-clockwise, 1 clockwise -->
  200 150 <!-- end at (200,150) -->
"/>
```



7. FARBEN

Nur ein **Teil** des elektromagnetischen Spektrums wird vom **Auge** wahrgenommen: Eine Wellenlänge von ca. 780 - 380 nm bzw. einer Frequenz von $3.8 \cdot 10^{14}$ bis $7.8 \cdot 10^{14}$ Hertz.

Wellenlänge · Frequenz = Lichtgeschw. ≈ 2.998 · 10⁸ m/s

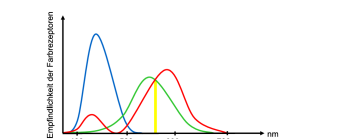
Spektralfarben bestehen aus Licht einer **einzigsten Wellenlänge**. In der **Natur** vorkommende Farben bestehen aus Licht, das aus **verschiedenen Wellenlängen** zusammengesetzt ist. Die Verteilung der Wellenlängen bezeichnet man als **Spektrum**.

7.1. CHARAKTERISIERUNG

- **hue:** Farbton, gegeben durch dominante Wellenlänge (hier A)
- **luminance:** Helligkeit A + B
- **saturation:** Sättigung A/(A + B)

Der Mensch sieht 100 Farbtöne, 50 Helligkeitsstufen und 20 Sättigungsgrade.

	Typ	Anzahl	Schwelle
S/W	Stäbchen	125'000'000	1 Photon
Farbe	Zäpfchen	5'000'000	100 Photonen



7.2. GRUNDFARBEN

Durch Mischen von Farben entstehen

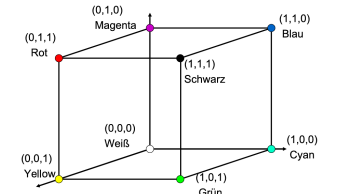
7.2.3. CMYK-Modell (subtraktiv)

Beim **Farbdruck** empfängt das Auge nur solche Anteile des weissen Lichtes, die **reflektiert** werden. Ein CMY-Tripel beschreibt, wie viel von den Grundfarben Cyan, Magenta und Yellow reflektiert bzw. von den Grundfarben Rot, Grün und Blau absorbiert wird (Weiss reflektiert alles, für Farben muss Rot/Grün/Blau absorbiert werden).

Beispiele:

- (0, 0, 0) absorbiert nichts, ergibt Weiss
- (0, 0, 1) absorbiert Blau, ergibt Gelb

Beispiel Farb-Subtraktion: (0, 1, 0) – (0, 0, 1) = (0, 1, 1)



Umrechnung

Verwendet zusammen mit dem CMY-Modell und dem RGB-Modell erfolgt in **Vektorschreibweise** über die Subtraktion.

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} - \begin{pmatrix} C \\ M \\ Y \end{pmatrix}, \quad \begin{pmatrix} C \\ M \\ Y \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} - \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

7.2.3. CMYK-Modell

Verwendet zusätzlich **schwarze Farbe**, den **Key**. Wird beim Drucken verwendet.

Umrechnungsnaherung mit Beispiel:

$$\begin{aligned} K &:= \min(C, M, Y) & K &:= \min(10, 15, 55) = 10 \\ C' &:= C - K & C' &:= 10 - 10 = 0 \\ M' &:= M - K & M' &:= 15 - 10 = 5 \\ Y' &:= Y - K & Y' &:= 55 - 10 = 45 \end{aligned}$$

Einer der Werte wird so immer 0.

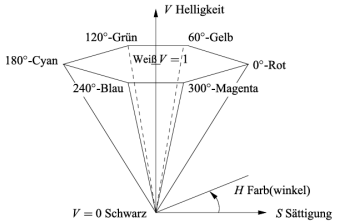
7.2.4. YUV-Modell

Wurde verwendet, um Fernsehbilder geräte-spezifisch entweder Farbe oder S/W anzuzeigen. Ein Farbwert wird durch ein YUV-Tripel beschrieben, wobei Y die Helligkeit (*Luminanz*) bezeichnet, und U, V die Farbdifferenzen (*Chromanz*).

7.2.5. HSV-Modell

Beschreibt jede Farbe durch das Tripel HSV.

Hue: Farbton, **Saturation:** Sättigung, **Value:** Helligkeit



Umrechnung RGB nach HSV

$$\begin{aligned} \text{rgb}(64, 128, 32) &= \left(\frac{1}{4}, \frac{1}{2}, \frac{1}{8} \right) \\ v &= \max(r, g, b) = 1/2 = 50\% \\ \min &:= \min(r, g, b) = 1/8 \\ s &= \frac{v - \min}{v} = \frac{4/8 - 1/8}{1/2} = \frac{3/8}{1/2} = \frac{3}{4} = 75\% \\ h &= \left(1 + \frac{v - r}{v - \min} \right) \cdot 60^\circ = \left(1 + \frac{2}{3} \right) \cdot 60^\circ = 100^\circ \\ \text{hsv} &= (100^\circ, 75\%, 50\%) \end{aligned}$$

Dominante Grundfarbe: Grün, weil v = g.

Schwächste Farbe: Blau, weil min = b → Farbe ist gelb-grünlich.

Umrechnung HSV nach RGB

$$\begin{aligned} \max(r, g, b) &= v \\ \min(r, g, b) &= v - s \cdot v \\ \text{mitte}(r, g, b) &= v - \left(\frac{h}{60^\circ} - 1 \right) \cdot (v - \min) \end{aligned}$$

8. 3D-TRANSFORMATIONEN

In 3D-Transformationen wird eine 4 × 4-Matrix mit homogenen Koordinaten verwendet.

8.1. TRANSLATION

Mit homogenen Koordinaten lässt sich der um Translationsvektor $\vec{t} = (t_x, t_y, t_z)^T$ verschobene Punkt $P = (x, y, z)$

$$(x', y', z') := (x + t_x, y + t_y, z + t_z)$$

in der folgenden Form darstellen:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} T(t_x, t_y, t_z) \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

8.2. SKALIERUNG

Gegeben sind drei Skalierungsfaktoren $s_x, s_y, s_z \neq 0$.

Der Fixpunkt der Skalierung liegt im Ursprung:

$$(x', y', z') := (x \cdot s_x, y \cdot s_y, z \cdot s_z)$$

Die daraus resultierende Transformationsmatrix lautet:

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Fixpunkt liegt nicht im Ursprung, sondern bei (Z_x, Z_y, Z_z) :

Zuerst Translation um $(-Z_x, -Z_y, -Z_z)$, dann Skalierung um (s_x, s_y, s_z) , dann Rücktranslation um (Z_x, Z_y, Z_z) :

$$T(Z_x, Z_y, Z_z) \cdot S(s_x, s_y, s_z) \cdot T(-Z_x, -Z_y, -Z_z)$$

8.3. ROTATION IM GEGENUHRZEIGERSINN

(Für Uhrzeigersinn müssen die Vorzeichen des Sinus vertauscht werden)

Rotation	Transformationsmatrix
Z-Achse	$R_z(\delta) = \begin{pmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
X-Achse	$R_x(\delta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\delta) & -\sin(\delta) & 0 \\ 0 & \sin(\delta) & \cos(\delta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
Y-Achse	$R_y(\delta) = \begin{pmatrix} \cos(\delta) & 0 & \sin(\delta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\delta) & 0 & \cos(\delta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Rotation um beliebige Achse

Rotationsachse stimmt nicht mit einer der Koordinatenachsen überein



0. Einheitsvektor $\vec{u}$ der Rotationsachse $\vec{v}$ (verläuft durch die Punkte P_1, P_2) berechnen und in Komponenten a, b, c zerlegen

$$\vec{v} = P_2 - P_1 = \begin{pmatrix} x_2 - x_1 \\ y_2 - y_1 \\ z_2 - z_1 \end{pmatrix}$$

$$|\vec{v}| = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

$$\vec{u} = \frac{\vec{v}}{|\vec{v}|} = \begin{pmatrix} a \\ b \\ c \end{pmatrix}, \quad a = \frac{x_2 - x_1}{|\vec{v}|}, \quad b = \frac{y_2 - y_1}{|\vec{v}|}, \quad c = \frac{z_2 - z_1}{|\vec{v}|}$$

1. Translation von Rotationsachse und Objekt, sodass **Rotationsachse durch den Ursprung** läuft:

$$\vec{v}' = (P'_1, P'_2)$$

$$P'_1 = (0, 0, 0), \quad P'_2 = (x_2 - x_1, y_2 - y_1, z_2 - z_1)$$

2. **Rotation der Rotationsachse** um die z-Achse in die xz-Ebene:

$$d = \sqrt{b^2 + c^2}, \quad \cos(\alpha) = \frac{c}{d}, \quad \sin(\alpha) = \frac{b}{d}$$

3. **Rotation der Rotationsachse** um die y-Achse in die z-Achse

$$\cos(\beta) = \cos(360^\circ - \beta) = d, \quad \sin(\beta) = -\sin(360^\circ - \beta) = -a$$

4. **Rotation des Objekts** um die z-Achse mit Winkel δ :

$$R_z(\delta) = \begin{pmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

5. **Rücktranslation** des gedrehten Objekts durch Anwendung der inversen Transformation der Schritte 3, 2 und 1.

8.4. TRANSFORMATION DER NORMALENVEKTOREN

Die Normalenvektoren müssen bei der Transformation von Objektpunkten ebenfalls abgebildet werden. Wenn diese Transformation eine **nicht-uniforme Skalierung** (Verzerrung) ist, bleiben die **Winkel** zwischen einzelnen Flächen **nicht erhalten**. Damit der Normalenvektor $\vec{n}$ trotzdem weiterhin senkrecht zur Fläche steht, muss dieser mit der **transponierten Inversen** der Transformationsmatrix M transformiert werden.

$$(M^{-1})^T \cdot \vec{n} = \vec{n'}$$

9. FRAKTALE

Fraktale sind geometrische Formen, die sich durch **Selbstähnlichkeit** auszeichnen: wenn man hineinzoomt, sieht das **Teilstück ähnlich oder gleich** wie das **gesamte Gebilde** aus.

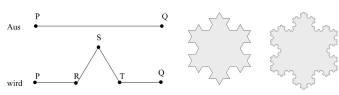
9.1. FRAKTALE DIMENSIONEN

Ein selbstähnliches Objekt hat **Dimension D**, falls es in **N identische Kopien** unterteilt werden kann, die jeweils **skaliert** sind mit dem Faktor $r = 1/N^{1/D}$.

Die Dimension D eines Fraktals lässt sich bestimmen, wenn N und r bekannt sind: $D = \log(N) / \log(\frac{1}{r})$.

9.2. KOCH'SCHE SCHNEEFLOCKE

R und T dritteln die Kante $\overline{PQ}$. S ist eine 60°-Drehung des Knotens T um das Zentrum R gegen den Uhrzeigersinn.



Die Koch'sche Schneeflocke hat Dimension $D = \frac{\log(4)}{\log(3)} = 1.2618...$, denn jeder Kantenzug besteht aus $N = 4$ Kopien, jeweils skaliert um den Faktor $r = \frac{1}{3}$.

9.3. LINDENMAYER-SYSTEME (L-SYSTEME)

Nicht-grafische Beschreibung mancher Fraktale.

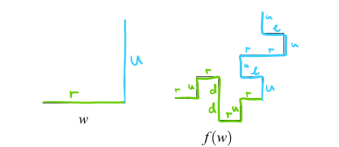
Alphabet: $\Sigma = r, u, l, d$ für right, up, left and down.

Regelmenge:

$$f = \begin{cases} r \rightarrow r u r d d r u r \\ u \rightarrow u l u r r u l u \\ l \rightarrow l d l u u l d l \\ d \rightarrow d r d l l d r d \end{cases}$$

Ausgehend vom Startwort w wird nun in jedem Iterationsschritt auf jedes Zeichen von w eine Regel aus f angewendet.

Beispiel: Sei $w = r$, dann ist $f(w) = r u r d d r u r u l u r u l u l u$



Die quadratische Koch-Kurve hat die Dimension $D = \frac{\log(8)}{\log(4)} = 1.5$, denn jeder Kantenzug besteht aus $N = 8$ Kopien, jeweils skaliert um den Faktor $r = \frac{1}{4}$.

9.4. BAUMSTRUKTUREN

Ausgangspunkt siehe rechts. An den Kanten BC und CD können jeweils weitere «Äste» mit Faktor 0.75/0.47 eingesetzt werden.



9.5. MANDELBROT-MENGE

Besteht aus komplexen Zahlen c , die beim Startwert $z = 0$ zu einer beschränkten Folge führen. Dann definiert man:

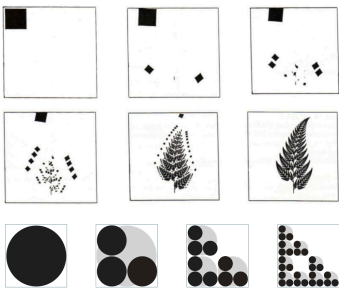
$$\text{farbe}(c) = \begin{cases} \text{schwarz,} & \text{wenn Folge beschränkt bleibt} \\ \text{weiss,} & \text{wenn Folge nicht beschränkt bleibt} \end{cases}$$

9.6. ITERIERTE FUNKTIONENSYSTEME

Sind in der Lage, mit **wenigen Regeln komplexe, natürlich aussehende Bilder** zu erzeugen.

Jede Transformation wird definiert durch eine 2 × 2-**Deformationsmatrix A**, einen **Translationsvektor b** und eine **Anwendungswahrscheinlichkeit w**. Dadurch wird ein Punkt x auf $A \cdot x + b$ mit Wahrscheinlichkeit w abgebildet.

Beispiele hierfür sind der **Farn** und das **Sierpinsky-Dreieck**:



10. MORPHOLOGIE

Morphologische Operationen **verändern die Form** von Bildobjekten (Dicke von Linien ändern, Vereinen/Trennen von Elementen, Unreinheiten eliminieren).

- **Operator:** Vereinigung, Differenz, Schnitt
- **f_s:** Menge von Pixeln
- **s:** Nachbarschaft, definiert durch Strukturelement

Voraussetzungen für Basisoperationen:

- Die Mengen f_s entsprechen Objekten oder Mustern im Bild
- Das Bild ist ein Binär- oder Grauwertbild
- Das Strukturelement (*Das Element, das über das Bild gelegt wird*) besteht aus 0 und 1, wobei die 1 die Pixel der Nachbarschaft definieren.

10.1. DILATATION

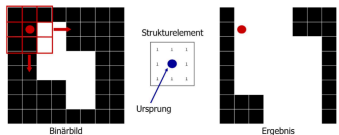
Das Strukturelement wird über jeden Pixel des Originalbildes gelegt und es wird eine logische **oder-Verknüpfung** zwischen dem Binärbild f und dem Strukturelement s durchgeführt: $g = f \oplus s$

Wirkung

Objekte werden **vergrößert** und können **verschmelzen**. Löcher können **geschlossen** werden, unregelmässige Objekte werden von der **Form her ausgeglichener**.

Beispiel

Das Strukturelement wird auf jeden Pixel angewendet. Da im Strukturelement alle Nachbarn 1 (weiss) sind, wird der Ursprung weiss eingefärbt, wenn einer der 8 Nachbarn bereits weiss ist.



10.2. EROSION

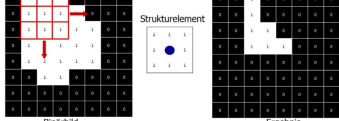
Das Strukturelement wird über jeden Pixel des Originalbildes gelegt und es wird eine logische **und-Verknüpfung** zwischen dem Binärbild f und dem Strukturelement s durchgeführt: $g = f \otimes s$

Wirkung

Objekte werden **verkleinert** und können **auseinanderfallen**. Löcher werden **grösser**, Objekte werden **gelöscht**, wenn sie **kleiner** als das Strukturelement sind.

Beispiel

Alle Pixel des Strukturelements müssen auf 1 (weiss) gesetzt sein, damit der Ursprung weiss bleibt.

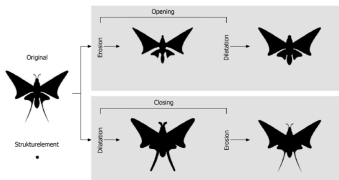


10.3. OPENING

Erosion mit anschliessender **Dilatation**: $A \circ B = (A \otimes B) \oplus B$
Wirkung: **Rundet Kanten**, **entfernt dünne Verbindungen** und dünne herausragende Strukturen.

10.4. CLOSING

Dilatation mit anschliessender **Erosion**: $A \bullet B = (A \oplus B) \otimes B$
Wirkung: **Rundet Konturen**, **verbindet nahe Objekte**, **füllt Löcher** und Einbuchtungen, die **kleiner** als das Strukturelement sind.

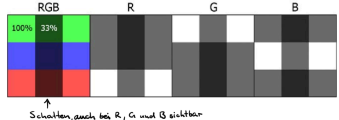


10.5. OBJEKTE FREISTELLEN

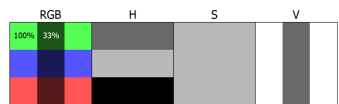
Zielsetzung: **Objekte** durch Erkennen eines Hintergrundes **freistellen**, **Hintergrund** durch andere Bildquelle **ersetzen**.

10.5.1. Hintergrundersetzung

Probleme: **Schatten** verfälschen Farbtöne. **Störungen** wie Rauschen oder Kompressionsartefakte sorgen für Ungleichmässigkeiten. **Farbtöne** und ihre Schattierungen sind in allen **RGB-Werten** kodiert.



Lösung: **Umwandlung** in HSV, um Farbton, Intensität und Sättigung zu trennen. Ermöglicht einfache Auswahl eines Farbtobereiches **unbeeinflusst** von Schattierungen.



Bereinigung der Selektionsmaske

Durch **Störungen** und schlechte **Farbtonverteilung** können **Fehler, Löcher** oder unschöne Kanten entstehen. Durch **Anwendung diverser Filter** oder **morphologischen Operationen** können diese Effekte **minimiert** werden.

Anwendungsschritte zur Hintergrundentfernung

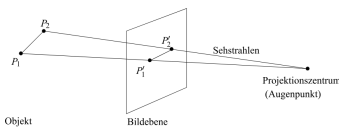
1. **Konvertieren** des Vordergrundes in **HSV**
2. **Erstellen der Alpha-Maske**
3. **Erstellung der Grauwertspreizung** (optional)
4. **Zusammenführung des V-Kanals** mit dem Hintergrund
5. **Morphologische Bildverbesserung**
6. **Vereinigen von Vorder- und Hintergrund**

Für jeden Pixel $[i, j]$ in Bild A
Falls Maske $[i, j]$ == true
Esetze Pixel $[i, j]$ in Bild A mit Pixel $[i, j]$ aus Bild B

11. PROJEKTION IN 3D

Das Abbilden von **dreidimensionalen Objekten** auf einer **zweidimensionalen Fläche** nennt sich **Projektion**. Gegeben sind das zu **projizierende Objekt**, die **Bildebene** und das **Projektionszentrum**.

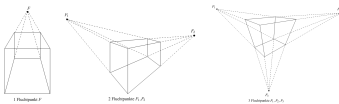
11.1. BILDEBENE



Ist der **Abstand** des Projektionszentrums von der Bildebene **endlich**, handelt es sich um eine **perspektivische Projektion** (Zentralprojektion), ansonsten um eine **Parallelprojektion** (Die beiden Projektionsstrahlen treffen sich nie).

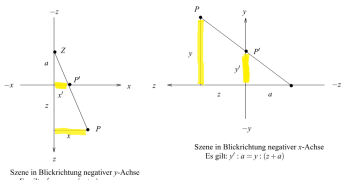
11.2. PERSPEKTIVISCHE PROJEKTION

Je nach Anzahl der geschnittenen Koordinatenachsen kann sie mit 1, 2 oder 3 **Fluchtpunkten F** umgesetzt werden.



11.2.1. Anwendung der Strahlensätze

- **Bildebene:** xy-Ebene
- **Projektionszentrum:** neg. z-Achse im Punkt $Z = (0, 0, -a)$
- **Gegeben:** $P = (x, y, z)$
- **Gesucht:** projizierter Bildpunkt $P' = (x', y', 0)$



Im Bild wird die Szene «von oben» und «von der Seite» betrachtet. Aufgrund der **Strahlensätze** erhält man die **Beziehung**

$$x' = \frac{x}{1 + z/a}, \quad y' = \frac{y}{1 + z/a}, \quad z' = 0.$$

Die **homogenen Koordinaten** des **projizierten Punktes** lauten

$$P' = \left(\frac{x}{w}, \frac{y}{w}, 0, 1 \right) \stackrel{w}{=} (x, y, 0, w) \text{ mit } w = 1 + z/a$$

Die **Transformationsmatrix** der perspektivischen Projektion ist:

$$P_{\text{persp}_{xy}}(-a) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{1}{a} & 1 \end{pmatrix}$$

11.3. PARALLELPROJEKTION

Stehen die Sehstrahlen im rechten Winkel zur Bildebene, liegt eine **orthogonale Projektion** vor. **Anwendung:** Grund-, An- und Seitenriss. Die **Transformationsmatrix** auf die xy-Ebene lautet (z-Komponente wird weggelassen):

$$P_{\text{ortho}_{xy}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

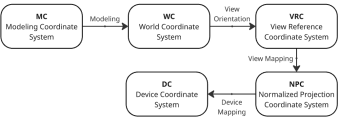
12. VIEWING PIPELINE

Die Abbildung dreidimensionaler Objekte auf dem Bildschirm wird in eine Reihe von Elementartransformationen zerlegt:

- **Modeling:** Konstruktion von komplexen Szenen aus elementaren Objekten
- **View Orientation:** Festlegen der Bildebene
- **View Mapping:** Projektion auf ein normiertes Gerät
- **Device Mapping:** Abbildung auf ein Ausgabegerät

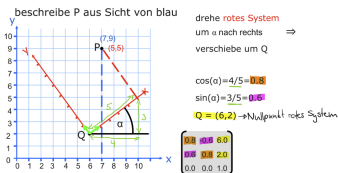
Schritte der Viewing Pipeline

- 1. **Modeling (MC → WC):** Beschreibe Polygonpunkte in Weltkoordinaten durch Translation, Rotation und Skalierung
- 2. **View Orientation (WC → VRC):** Überführe Szene in Kameraperspektive durch Wechsel des Koordinatensystems. Dann ist xz -Ebene = Bildebene und das Auge liegt in $z = VRP$.
- 3. **View Mapping (VRC → NPC):** Transformiere Szene in Einheitswürfel, damit Vorder-/Rückseite der Front-/Backplane entsprechen.
- 4. **Device Mapping (NPC → DC):** Projiziere Szene auf Bildschirm. x, y liefert Koordinaten, z Tiefeninformation.



12.3. KOORDINATENSYSTEMWECHSEL

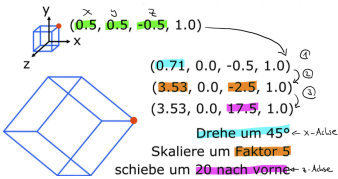
Konvertierung von lokalem zu globalen Koordinatensystem.



Matrix transformation for point P(7,5) to Q(6,2):

$$\begin{pmatrix} 0.8 & -0.6 & 6.0 \\ 0.6 & 0.8 & 2.0 \\ 0.0 & 0.0 & 1.0 \end{pmatrix} \cdot \begin{pmatrix} 7.0 \\ 5.0 \\ 1.0 \end{pmatrix} = \begin{pmatrix} 7.0 \\ 9.0 \\ 1.0 \end{pmatrix}$$

12.3.1. Modeling
Das Anordnen von Objekten aus dem Modellkoordinatensystem zu einer Szene im Weltkoordinatensystem. Die Objekte erhalten aus der Grundform ihre individuelle Grösse, Orientierung und Position durch Translation, Rotation und Skalierung.



12.3.2. View Orientation

Zur Abbildung einer dreidimensionalen Szene aus den Weltkoordinaten auf dem Bildschirm muss eine Betrachtersicht (View Orientation) definiert werden. Diese besteht aus den Parametern:

- PRP: Betrachterstandpunkt (Projection Reference Point)
- VRP: Blickrichtung (View Reference Point)
- VUV/VUP: vertikale Orientierung (View Up Vector/View Up Point)
- Blickwinkel: Brennweite

Wenn die Achsen der Kamera mit $U - V - N$ und der Nullpunkt der Kamera mit VRP bezeichnet wird (Siehe Bild in Kapitel 12.1. «Die Synthetische Kamera»), ist die Transformation von lokale in globale Koordinaten wie folgt definiert:

$$M = \begin{pmatrix} U_x & V_x & N_x & VRP_x \\ U_y & V_y & N_y & VRP_y \\ U_z & V_z & N_z & VRP_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Damit die global definierte Szene aus Kamerasicht angeschaut wird (Wechsel von globalen zu lokalen Koordinaten – also umgekehrt zur obigen Rechnung.), muss auf die Modellgeometrie die inverse Transformation $T = M^{-1}$ angewendet werden.

$$M^{-1} = \begin{pmatrix} U_x & U_y & U_z & -(\vec{U} \cdot VRP) \\ V_x & V_y & V_z & -(\vec{V} \cdot VRP) \\ N_x & N_y & N_z & -(\vec{N} \cdot VRP) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Beispielaufgabe Kameratransformation

Kamera schaut von $p1 = (2, 2, 0)$ auf $p2 = (1, 1, 0)$. Die x -Achse sei parallel zur globalen xy Ebene. Die z -Achse der Kamera zeigt in Richtung Blickrichtung ($p1 \rightarrow p2$). Berechne die Transformationsmatrix der Kamera (M) und die Transformationsmatrix, mit der globale Koordinaten in das Kamerakordinatensystem transformiert werden (M^{-1}).

Es gilt: $N = Z$ -Achse der Kamera, $V = Y$ -Achse der Kamera, $U = X$ -Achse der Kamera. $PRP =$ Kamerapunkt, $VRP =$ Angeschauter Punkt.

N, V, U kann von der Zeichnung abgelesen werden. Zeigt Veränderung der globalen Koordinaten (+1 zeigt in die globale z -Richtung, -1 zeigt in globale z -Richtung).

Normalisieren mit $\sqrt{x^2 + y^2 + z^2}$: U zu $\vec{U}$

$$\vec{U} = \frac{1}{\sqrt{1^2 + 1^2 + 0^2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

Inverse Werte von M^{-1} berechnen:

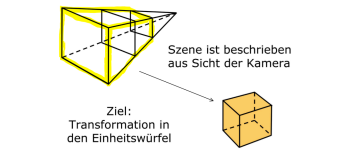
$$-(\vec{U} \cdot VRP) = -\left(\frac{1}{\sqrt{2}} \cdot \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}\right) = -\left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{2}} + 0\right) = 0$$

Normalisierte Werte in M und M^{-1} eintragen:

$$M = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad M^{-1} = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

12.3.3. View Mapping

View Volume: Bildraum, der durch das View Window und die gewählte Projektion definiert wird. Das Mapping des View Volume auf den Einheitswürfel, dessen Vorder- und Rückseite die Front- bzw. die Backplane darstellen, dient der Effizienzsteigerung des Algorithmus in späteren Schritten und erleichtert zudem die Projektion der Szene auf die Bildebene: Anstelle mehrerer unterschiedlicher Projektionen muss so nur noch die orthogonale Parallelprojektion auf die Ebene $z = 0$ durchgeführt werden.

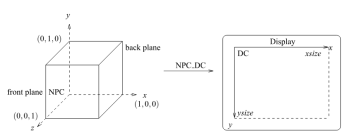


Dieses Mapping wird in einigen komplizierten mathematischen Schritten durchgeführt:

- Verschiebung des PRPs in den Ursprung: $z := z - d$
- Spiegelung an der xy -Ebene: $z := -z$
- Überführung in Pyramidenstumpf (90° Winkel von PRP zu y_{min} und y_{max})
- Die regelmässige Pyramide (45°, 90°, 45°) kann nun in den Einheitswürfel transformiert werden.

12.3.4. Device Mapping

Die Abbildung muss nun die x - und y -Koordinaten aus dem NPC so in die Bildschirmkoordinaten DC (Device Coordinate System) transformieren, dass eine anschliessende Rundung die ganzzahligen Koordinaten der Pixel ergibt.



Auch ist DC oft ein linkshändiges Koordinatensystem (y -Achse zeigt nach unten, der Ursprung ist oben links), dafür muss die Abbildung ebenfalls noch angepasst werden.

Die Transformationsmatrix entspricht einer Skalierung um den Vektor ($xsizex, -ysizey, 1$) konkateniert mit einer Translation des Ursprungs in die linke untere Ecke des Bildschirms ($0, ysize, 0$).

$$T_{NPC,DC} = \begin{pmatrix} xsizex & 0 & 0 & 0 \\ 0 & -ysizey & 0 & ysize \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

12.3.5. Clipping

Jedes Polygon, dass die Viewing Pipeline durchläuft, kostet Rechenaufwand. Deshalb macht es Sinn, den unsichtbaren Teil einer Szene so früh wie möglich loszuwerden. Es gibt verschiedene Optionen dafür.

Clipping im WC / NPC

Im WC muss an den sechs Flächen des Frustum (Kegelstumpf) geclickt werden, während im NPC an den sechs «einfachen» Flächen des Einheitswürfels mit Cohen-Sutherland

(Kapitel 4.1. «Cohen-Sutherland Algorithmus») geclickt werden kann. Clipping im WC spart eine Transformation aller Punkte, dafür ist im NPC das eigentliche Clipping und die Schnittpunktberechnung einfacher.

Umgebungsclipping

Bietet Effizienzsteigerung des Clippings, indem ein Cluster von mehreren, komplexen Objekten mit einem grossen Quader umgeben wird. Ergibt ein erster Clipping-Test, dass dieser Quader ausserhalb des Frustum liegt, erübrigen sich die Clipping-Abfragen seiner inneren Objekte.

13. OBJEKTE IN 3D

Für 3-dimensionale Objekte gibt es mehrere Möglichkeiten der Repräsentation (d.h. Definition des Objekts) und der Darstellung (d.h. Projektion des Objekts auf den Bildschirm).

Repräsentation:

- Elementarobjekt mit Definitionspunkten (Mehrfachverwendung bestehender Geometrie durch Transformationen, veraltet)
- Drahtmodell (Liste von Kanten, skizziert Umrisse, veraltet)
- Flächenmodell mit Punkt- und Flächenliste und Normalen
- CSC (constructive solid geometry) mit mengentheoretischer Verknüpfung von Elementarobjekten (Ein Objekt besteht aus Addition/ Subtraktion mehrerer Elemente, z.B. Rohr = Grosser Zylinder - kleiner Zylinder).

Darstellung:

- Punktmodell (zeigt nur Eckpunkte eines Objekts an)
- Drahtmodell mit sämtlichen Kanten (Punkte sind verbunden)
- Drahtmodell mit Entfernung verdeckter Kanten
- Flächenmodell mit Einfärbung, ohne abgewandte Flächen
- Flächenmodell mit Einfärbung, ohne verdeckte Teile von Flächen
- Flächenmodell mit Einfärbung, ohne verdeckte Teile von Flächen, mit Beleuchtungsmodell
- Körpermodell mit Berechnung von Schattenbildung, Spiegelungen und Brechungen.

13.1. FLÄCHENMODELL

Objekte werden durch approximierte oder analytische Flächen, z.B. durch eine Liste von konvexen Polygonen, repräsentiert. Beim Würfel verbinden die Kanten die Eckpunkte, bei einer Kugel werden die Längen- und Breitenkreise durch n -Ecke angenähert, wobei mit n die Qualität der Approximation steigt.

Beispiel Tetraeder:

Punktliste	Flächenliste
$P_1: (x_1, y_1, z_1)$	$F_1: P_1, P_2, P_4$
$P_2: (x_2, y_2, z_2)$	$F_2: P_1, P_4, P_3$
$P_3: (x_3, y_3, z_3)$	$F_3: P_1, P_2, P_3$
$P_4: (x_4, y_4, z_4)$	$F_4: P_4, P_2, P_3$

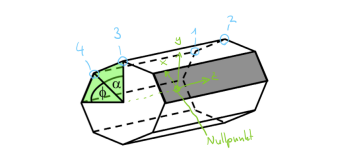
13.2. POLYEDER

Ein Polyeder ist ein Körper, dessen Oberfläche aus ebenen Flächen besteht. Datenstruktur, um Polyeder zu beschreiben: Punkte, Kanten, Flächen, Normale, Farbe, Materialeigenschaften, Textur, BumpMap.

13.2.1. Zylinder

Ein Zylinder kann mit 4 Punkten komplett beschrieben werden.

#	Eckpunkt	Normalenvektor
1	$(\cos(\phi), \sin(\phi), +1)$	$(\cos(\phi), \sin(\phi), 0)$
2	$(\cos(\phi + \alpha), \sin(\phi + \alpha), +1)$	$(\cos(\phi + \alpha), \sin(\phi + \alpha), 0)$
3	$(\cos(\phi + \alpha), \sin(\phi + \alpha), -1)$	$(\cos(\phi + \alpha), \sin(\phi + \alpha), 0)$
4	$(\cos(\phi), \sin(\phi), -1)$	$(\cos(\phi), \sin(\phi), 0)$

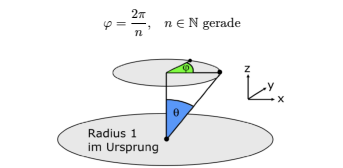


13.2.2. Kugel

Die Oberfläche einer Kugel mit Radius 1 kann beschrieben werden durch

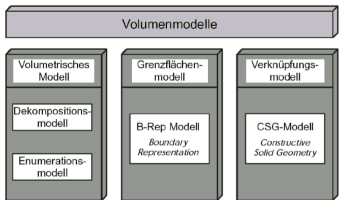
$$(\sin(\theta) \cdot \cos(\varphi), \sin(\theta) \cdot \sin(\varphi), \cos(\theta)), \quad 0 \leq \varphi < 2\pi, \quad 0 < \theta < \pi$$

Zur Approximation durch Flächen wird der Vollwinkel in n Teile zerlegt:



13.3. VOLUMENMODELLE

Volumenmodelle bilden Oberfläche und Inhalt eines dreidimensionalen Objekts ab und beschreiben den dreidimensionalen Körper eindeutig.



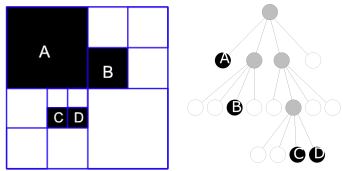
13.3.1. Volumetrische Modelle

In volumetrischen Modellen werden Objekte durch diskrete Volumenelemente repräsentiert.

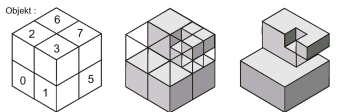
Enumerationsmodelle: Alles wird mit identischen, aber verschieden skalierten geometrischen Elementen wie z.B. Würfel gefüllt. Beispiel: Octree Dekompositionsmodelle: Stellen ein Objekt als Zusammensetzung einzelner Zellen dar, die verschieden geformt sind. Werden hauptsächlich für FEM-Anwendungen verwendet (Finite Elemente Methode).

13.3.2. Octree

Eignet sich zur Verwaltung der räumlichen Anordnung von Objekten im dreidimensionalen Raum. Wenn ein Gebiet nicht komplett gefüllt ist, wird es geverteilt. Die Viertel werden im Uhrzeigersinn abgearbeitet, Start oben links. (Zumindest in diesem Beispiel). Grauer Knoten: Enthält teilweise Objekte.

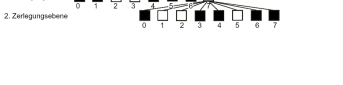


Das Konzept wird Octree genannt, weil ein Würfel jeweils in 8 Teilstücke geteilt wird, und Nodes im Baum somit jeweils 8 Child Nodes haben.



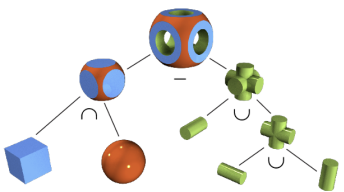
Octree-Darstellung:

- 1. Zerlegungsebene
- 2. Zerlegungsebene



13.4. CSG (CONSTRUCTIVE SOLID GEOMETRY)

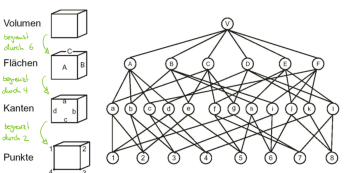
Erzeugt durch regularisierte Mengenoperationen mit anderen Objekten: $\cap$ = Vereinigung, $\cup$ = Durchschnitt, $\setminus$ = Differenz. Die Wurzel repräsentiert das resultierende Objekt, welches aus den Blättern unter Anwendung der Operationen konstruiert werden kann.



13.5. BREP (BOUNDARY-REPRESENTATION)

Wird in heutigen CAD-Systemen oft verwendet. Ein Körper wird durch Flächen beschrieben. Durch die Orientierung der Normalvektoren kann eindeutig zwischen Objekt-innerem und Objekt-äusserem unterschieden werden. Flächen werden wiederum durch Kanten und Kanten durch Punkte berandet.

Die Datenstruktur ist durch folgende Charakteristika gekennzeichnet: – Topologie und Geometrie sind getrennt dargestellt: Topologie in der Hierarchie und die Geometrie in den Knoten. – Die Topologie hat im einfachsten Fall nur vier Hierarchiestufen.



13.5.1. Euler-Gleichung

Veränderungen des Graphen sind über Euler-Operatoren möglich. Diese trifft eine Aussage über die Anzahl von Ecken, Kanten, etc. in einem physikalischen Objekt.

Kürzel	Objekt
E / V	Ecken / Vertices
K / E	Kanten / Edges
S / F	Seiten / Faces
I / R	Innere Zyklen / Rings (Löcher innerhalb Flächen)
Z / S	Komponenten / Shells (Separate Körper, meist 1)
G / G	Löcher / Genus (Löcher durch ganzes 3D Objekt)

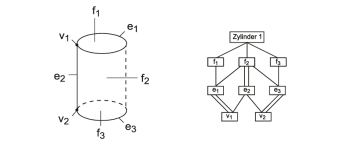
Gleichung (deutsch): $E - K + S - I = 2(Z - G)$
Gleichung (englisch): $V - E + F - R = 2(S - G)$

Jedes Volumenmodell muss diese Gleichung erfüllen. Zusätzlich zu der Euler-Gleichung müssen BREPs folgende Voraussetzungen erfüllen:

- Jede Kante trennt genau zwei Flächen (evtl. auch gewölbte Fläche)
- Um jede Ecke existiert ein geschlossener Ring von Flächen – Flächen können sich nur an einer gemeinsamen Ecke oder Kante schneiden

Beispiel Zylinder:

Ein Zylinder kann wie folgt beschrieben werden:



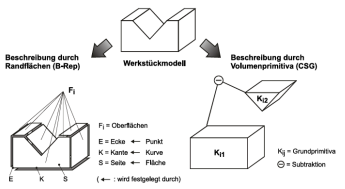
Er hat also 2 Ecken v , 3 Kanten e , 3 Flächen f , 0 Innere Zyklen, 1 Komponente und 0 Löcher. Wenn wir diese Zahlen in die Euler-Gleichung einfügen, erhalten wir folgende Gleichung:

$$E - K + S - I = 2(Z - G) \quad 2 - 3 + 3 - 0 = 2(1 - 0) \Rightarrow 2 = 2 \checkmark$$

Achtung: Bei halben Löchern muss keine separate Kante mehr verwendet werden, die Abbruchkanten des Halbkreises reichen.

13.6. KONSTRUIEREN MIT BREP VS. CSG

Das gleiche Modell kann durch Randflächen (BREP) oder durch Volumenprimitive (CSG) beschrieben werden:



14. CULLING

Culling ist das Entfernen von unsichtbaren Kanten, Flächen und Objekten. Es gibt zwei Arten von Culling:

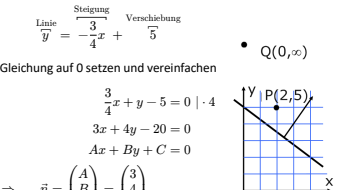
- Objektraum: Arbeitet mit Weltkoordinaten, vergleicht Objekte
- Bildraum: Arbeitet mit Device-Koordinaten, vergleicht Pixel

14.1. BACK-FACE REMOVAL

Das Entfernen von Flächen, die dem Betrachter abgewandt sind. Nützlich, weil diese nicht sichtbar sein sollten, aber auch, weil sich die weitere benötigte Rechenleistung dadurch ca. halbiert.

14.1.1. Geradengleichung

Die Linie y ist sichtbar vom Punkt P , wenn $\vec{p} \cdot \vec{n} > C \geq 0$ und sichtbar von Punkt Q , falls $B \geq 0$. Beispiel:

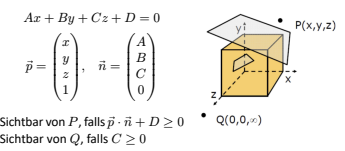


$$\vec{p} = \begin{pmatrix} 2 \\ 5 \end{pmatrix}, \quad \vec{n} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \quad \vec{p} \cdot \vec{n} = \begin{pmatrix} 2 \cdot 3 \\ 5 \cdot 1 \end{pmatrix} = \begin{pmatrix} 6 \\ 5 \end{pmatrix}$$

Sichtbar von P? $6 + 20 + (-20) = 6 \geq 0 \Rightarrow$ sichtbar.
Sichtbar von Q? $B = 4 \geq 0 \Rightarrow$ sichtbar.

14.1.2. Ebenengleichung

Ähnlich wie Geradengleichung, aber für 3D.



Wenn $\vec{n}$ der Normalenvektor der Fläche und $\vec{a}$ ein Eckpunkt ist, dann kann die Gleichung der Ebene, in der die Fläche liegt, in der Hesseschen Normalform bestimmt werden.

$$\vec{p} \cdot \vec{n} - \vec{a} \cdot \vec{n} = e$$

Beim Einsetzen verschiedener Punkte $\vec{p}$ ergeben sich unterschiedliche Werte für e .

- $e = 0$: $\vec{p}$ liegt in der Ebene
- $e > 0$: $\vec{p}$ befindet sich ausserhalb, d.h. ist die Fläche von $\vec{p}$ aus sichtbar.
- $e < 0$: $\vec{p}$ befindet sich innerhalb, d.h. ist die Fläche von $\vec{p}$ aus unsichtbar.

14.1.3. Back Face Culling

Für jede Polygonfläche berechne die z -Komponente der Flächennormalen im NPC. Falls $E_{\text{genuss}} \leq 0 \Rightarrow$ Face nicht sichtbar, Fläche kann vergessen werden.

$$\begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \cdot$$

14.2. HIDDEN SURFACE REMOVAL (HSR)

Das Entfernen von nicht sichtbaren Flächen. Hierfür gibt es verschiedene Algorithmen.

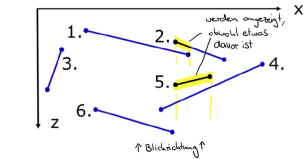
14.2.1. Painter Algorithm

Ordnet die zu visualisierenden Flächen und zeichnet sie dann von hinten nach vorne. Die weiter entfernten Flächen werden immer wieder übermalt.

Ablauf:

- 1. Ordne alle Polygone nach kleinstem z-Wert
- 2. Polygone mit überlappendem z-Ausdehnung ggf. umordnen
- 3. Ausgabe der Flächen von hinten nach vorne.

Problem: Überlappende Flächen werden nicht korrekt angezeigt, müssten zerschnitten werden.



14.2.2. z-Buffer Algorithm

Löst das Problem des Painter-Algorithmus. Benötigt zwei 2-dimensionale Arrays als Buffer: Einmal den z-Buffer, der für jedes Pixel den z-Wert des in diesem Punkt dem Betrachter am nächsten liegenden Objekts enthält (grosser z-Buffer -> näher am Betrachter), und einmal den Frame Buffer, welcher den Farbwert jedes Pixels speichert.

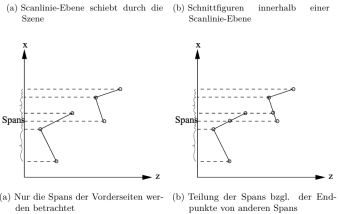
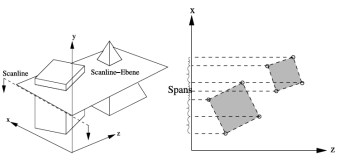
For each area:
Für jedes Pixel (x,y) on this area:
calculate color c and Depth z
if z > depth[x,y]
Add color c at (x,y) in the Frame Buffer and set depth[x,y] to z

Wird vor diesem Algorithmus Back Face Culling durchgeführt, wird er effizienter. Das Ergebnis bleibt aber das Gleiche.

Problem: Nicht sehr effizient, weil er für jedes Pixel eine Scanline die z-Koordinate berechnen muss. Es kann auch sein, dass ein bereits gesetztes Pixel später überschrieben wird.

14.2.3. Span-Buffer Algorithm

Löst das Effizienz-Problem des z-Buffer Algorithmus. Eine Scanline durchläuft das Bild und zerfällt in Abschnitte (Spans). Pro Span ist genau ein Polygon sichtbar.
Spans: Nur Vorderflächen nach x-Koordinate sortieren, ggf. Flächen zerschneiden und vereinen.



Ist effizienter, weil kein Pixel mehrfach eingefärbt und die Tiefe nicht überprüft werden muss, dafür muss zusätzliche Arbeit für die Ermittlung der Spans aufgewendet werden.

Problem: Geheiligtes Durchdringen/Kreuzen wird nicht erkannt. Bei hohen Polygonanzahl weniger effizient als Z-Buffer.

14.2.4. Binary Space Partition Tree

Analysiert die Lage der Objekte untereinander mithilfe einer Baumstruktur. Ist unabhängig von der Kamera, die Root-Kante kann beliebig gewählt werden. Sehr teuer in der Berechnung ($O(n^2)$), nur für statische Szenen (z.B. Kamerafahrt in Cuscene).

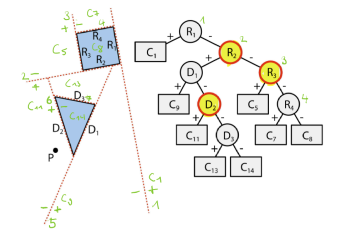
Der BSP-Tree speichert die Objekt-Beziehungen. Jeder innere Knoten repräsentiert eine Polygonfläche F, welche die Szene aufteilt in

- vorderen Teil: F ist hier sichtbar
- hinterer Teil: F ist hier nicht sichtbar



Ablauf zur Berechnung eines BSP-Tree:

- Alle Kanten aller Objekte im Raum nummerieren (R1-R4, D1-D3)
- Entlang der ersten Kante des ersten Objektes eine Linie ziehen (Linie C1 entlang Kante R1, L1). Markieren, welche Seite der Linie vor der Fläche (+) und welche dahinter (-) liegt.
- Kante und Linie in Tree eintragen. Auf der äusseren Seite des Objektes ist der sichtbare Teilbaum, auf der anderen der nicht sichtbare. Wenn eine Seite gar keinen Inhalt hat, wird dies im Baum mit einem rechteckigen Feld markiert.
- Mit der zweiten Seite des ersten Objektes fortfahren (R2).
- So lange wiederholen, bis alle Kanten im Baum angegliedert sind und alle Blätter ein eckiges Feld sind.



Im Bild sind die Flächen gelb markiert, die in Richtung P zeigen - also diejenigen Kanten, deren +Seite dem Punkt P zugewandt sind. In diesem Fall werden R2 und R3 jedoch von D2 überdeckt und sind deshalb trotzdem nicht sichtbar.

Flächen, die auf der selben Seite liegen wie der Augenpunkt...
- können Flächen auf der anderen Seite verdecken
- können von Flächen auf der anderen Seite nicht verdeckt werden

Achtung: Wenn eine der Trennebenen (gezeichnete Linien) durch eine andere Fläche durchgeht, muss die Fläche am Schnittpunkt der Linie in zwei Hälften geteilt werden.

15. BELEUCHTUNG

Es gibt zwei Beleuchtungsmodelle: lokal und global. Damit Beleuchtung auf Objekte angewendet werden kann, müssen diese zuerst trianguliert (in Dreiecke umgewandelt) werden. Diese eignen sich ausgezeichnet für Scanline-Verfahren, da für jede Bildschirmzeile maximal zwei Schnittpunkte mit den Dreiecksanten auftreten.

- Konvexe Polygone: Von einem beliebigen Eckpunkt aus zu allen nicht benachbarten Eckpunkten Diagonale ziehen.
- Konkave Polygone: Etwas komplizierter, hierzu gibt es Libraries

In die Berechnung der Farbe eines Pixels fliessen mehrere Faktoren ein: Materialeigenschaften des Objekts, Augenpunkt des Betrachters, Normalenvektor des Objekts, Positionierung und Art der Lichtquellen.

15.1. LICHTQUELLEN

Es gibt verschiedene Arten von Lichtquellen:

- Umgebungslicht / Ambient Light: Hat weder Position noch Richtung. Generiert keinen Schatten. Wird mit konstanter Intensität I_a beschrieben (z.B. «Fullbright» in Source Engine).
- Gerichtetes Licht / Directed Light: Hat keine Position, aber eine Richtung. Wird mit Intensität I_a und Lichtrichtung L_a beschrieben (z.B. Sonnenlicht).
- Punktlicht / Point Light: Hat eine Position P, aber keine bevorzugte Richtung (Strahlt vom Punkt in alle Richtungen aus). Hat eine Anfangs-Intensität I_0 , welche mit Entfernung abnimmt.

$$I(r) = \frac{I_0}{C_1 + C_2 \cdot r}$$

$$\begin{matrix} r \geq 0 & C_2 \geq 0 & C_1 \geq 1 \\ \text{Abstand zur Lichtquelle} & \text{Abschwächungskoeffizient} & \text{(verhindert zu kleinen Nennern)} \end{matrix}$$

- Strahler / Spot Light: Hat Position P, Lichtrichtung L, Intensität I_0 , Abschwächungskoeffizienten C_1, C_2 , Abstrahlwinkel α und eine Konzentrationsexponenten c . Die Intensität bei Richtung $r = \cos(r, L)^c$, wobei (r, L) der Zwischenwinkel zwischen r und L darstellt. Je grösser c , desto stärker ist die Intensität im Zentrum des Lichtkegels gebündelt (z.B. Nachtsichtlampe).

15.1.1. Gesamtbeleuchtung auf ein Pixel

Setzt sich zusammen aus ambientem Licht $\bar{C}_a$, diffus reflektiertem gerichtetem Licht $\bar{C}_d$ und spekulär reflektiertem gerichtetem Licht $\bar{C}_s$ (I_a = Lichtquelle).

$$\bar{C} = \bar{C}_a + \sum_{i=1}^n \bar{C}_{d_i} + \sum_{i=1}^n \bar{C}_{s_i}$$

Die Gesamtbeleuchtung muss für jede Farbe einzeln berechnet werden.

15.2. REFLEXION

Das Reflexionsverhalten eines Körpers wird durch folgende Eigenschaften bestimmt:
- k_a : ambierter Reflexionskoeffizient (wie stark reflektiert amb. Licht)
- k_d : diffuser Reflexionskoeffizient (wie stark reflektiert diffuses Licht)
- k_s : spekulärer Reflexionskoeffizient (wie stark reflektiert Punktlicht)
- O_d : diffuse Objektfarbe (durch Spiegelung erzeugt)
- O_s : spekuläre Objektfarbe (durch Spiegelung erzeugt)
- O_c : spekulärer Exponent

15.2.1. Ambiente Reflexion

Grundhelligkeit eines Objekts.

$$\bar{C}_a = \frac{k_a}{\text{ambierter Reflexionskoeffizient}} \cdot \frac{I_a}{\text{Intensität des ambienten Lichts}} \cdot \frac{O_d}{\text{diffuse Objektfarbe}}$$

15.2.2. Diffuse Reflexion

Vom Objekt diffus reflektiertes Licht (Lichtstreuung), ist überall gleichmässig sichtbar.

$$\bar{C}_d = \frac{k_d}{\text{diffuser Reflexionskoeffizient}} \cdot \frac{I_d}{\text{Intensität des einfallenden Lichts}} \cdot \frac{O_d}{\text{diffuse Objektfarbe}}$$

15.2.3. Spekulare Reflexion

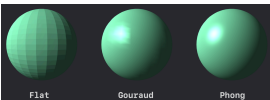
Vom Objekt gespiegeltes Licht R , nur in bestimmte Richtung sichtbar.

$$\bar{C}_s = \frac{k_s}{\text{spekulärer Reflexionskoeffizient}} \cdot \frac{I_s}{\text{Intensität des einfallenden Lichts}} \cdot \frac{O_s}{\text{spekuläre Objektfarbe}} \cdot \cos^2(\angle R/A)^{O_c}$$

15.2.4. Materialeigenschaften

- Objekte sollten nicht mehr abstrahlen als empfangen:
 $0 \leq k_a, k_d, k_s \leq 1$
 $k_a + k_d + k_s \leq 1$
- kontrastarm: Ambierter Teil ist am stärksten: $k_a \gg k_d, k_s$
- matt: Diffuser Teil ist viel grösser als Spekularteil: $k_d \gg k_s$
- spiegelnd: Spekularteil ist grösser als Diffusteil: $k_s \gg k_d$

15.3. SCHATTIERUNG (SHADING)



15.3.1. Flat Shading

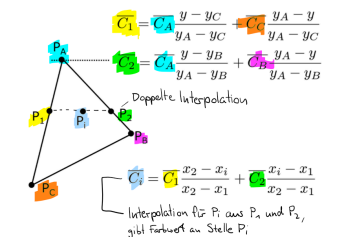
Pro Dreieck/Polygon eine Farbe. Eckpunkte im WC beleuchten, Farb-Mittelwert C_i der Eckpunkte berechnen, diese Farbe für alle Pixel im Polygon verwenden. Kanten zwischen Polygonen sind mit dieser Variante sichtbar.

$$\bar{C}_i = \frac{\bar{C}_A + \bar{C}_B + \bar{C}_C}{3}$$

Diagramm eines Dreiecks mit Eckpunkten $P_A [200,160,10]$, $P_B [200,70,10]$, $P_C [200,20,10]$ und $P_D [200,10,10]$. Berechnung des Mittelwerts: $\frac{30+160+10}{3} = 70$ für die x-Koordinate.

15.3.2. Gouraud Shading

Interpolation der Farwerte C_i . Die Farwerte der Eckpunkte $\bar{C}_A, \bar{C}_B, \bar{C}_C$ werden fix gesetzt und der Rest wird mit Interpolation gefüllt. $y = y$ -Wert der Scanline.



15.3.3. Phong Shading

Aufwändigste Version. Interpoliert pro Scanline die Anfangs- und die Endnormale. Pro Pixel wird ebenfalls die Normale interpoliert und daraus der Farbwert berechnet. Im Vergleich zu Gouraud Shading ist bei Phong wirklich jeder Pixel unterschiedlich.

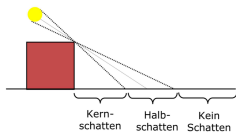
15.4. SCHATTEN

Von der Lichtquelle nicht sichtbare Pixel. Um zu berechnen, ob ein Pixel im Schatten liegt oder nicht, kann ein beliebiger Hidden-Surface-Removal-Algorithmus verwendet werden.

- Phase 1: Rendere Bild aus Position der Lichtquelle L in einem Schattentiefenpuffer s_{side}
- Phase 2: Rendere Bild aus Position des Betrachters mit modifiziertem Z-Buffer-Algorithmus: falls Pixel $P(x, y, z)$ sichtbar ist, transformiere P in den Koordinatenraum von Phase 1 zu $P'(x', y', z')$.

Falls $z' < s_{side}[x', y']$, dann ist P im Schatten von L.

Falls $z' \geq s_{side}[x', y']$, dann ist P nicht im Schatten von L.

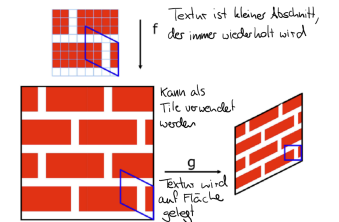


16. TEXTURING

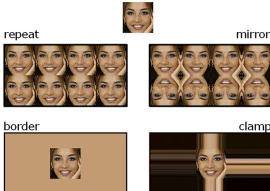
Verfahren, bei denen das Aussehen einer Fläche verändert wird. Zur realistischen Gestaltung verwendet man ein zweidimensionales Musterfeld (Texture Map), bestehend aus Texeln (Texturepixeln), aus denen für jedes Pixel die Farbe ermittelt werden kann. Die Textur wird zuerst an die Geometrie des Objekts angepasst und anschliessend darauf gemappt.

16.1. PHASEN DES TEXTURE MAPPING

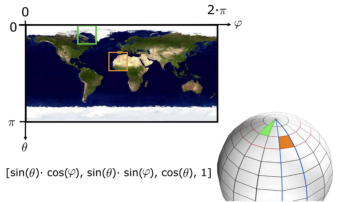
- Raumkoordinaten des Flächenpunktes berechnen (x', y', z')
- Zugehörige Flächenkoordinaten berechnen $\Rightarrow (x, y)$
- Abbildung in den Parameterraum durchführen $\Rightarrow (u, v)$
- Texturkoordinaten berechnen (Korrespondenzpunkt berücksichtigen)
- Texturwerte ermitteln
- Erscheinungsbild mit dem Texturwert modifizieren.



16.1.1. Korrespondenzfunktion

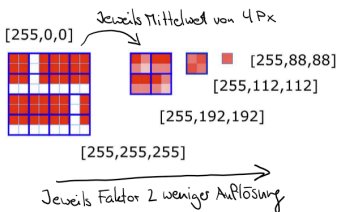


16.1.2. Sphärische Projektion



16.2. TEXTUR-ARTEFAKTE / MIP MAPPING

multum in parvo mapping: Viel in wenig. Halte verschiedene bereits vorberechnete Textur-Auflösungen für verschiedene Level of Detail (LOD) bereit.



$$x = (1-t) \cdot x_0 + t \cdot x_1, \text{ gleiche Formel auch für y und z}$$
$$(x-a)^2 + (y-b)^2 + (z-c)^2 = r^2$$

Einsetzen liefert quadratische Gleichung in t. Resultat: 0 (nicht auf Kugel), 1 (tangential auf Kugel) oder 2 Schnittpunkte (x, y, z)

16.3. ALGORITHMEN

Arten der Texturanwendung: statisch: Texture Map normal anwenden, mit Störungen: Veränderung der Textur durch Noise, sieht dann natürlicher aus, prozedural: Mit bestimmten Algorithmen können Muster wie Marmorierung erstellt werden. Light Map: Pro Face die Beleuchtung vorberechnen und in Light Map ablegen.

Shadow Map: Berechne z-Buffer aus Sicht der Lichtquelle und lege in Shadow Map ab. Modifiziere Pixelfarbe mit Hilfe der Shadow Map. Artefakte können je nach Auflösung des z-Buffers entstehen. Oft zusammen mit Light Map generiert.

Alpha Mapping: Für teilweise durchsichtige Elemente wie z.B. Laub an Baum. Textur enthält Alphawerte: 0 = durchsichtig, $0 < x < 255$ = teilweise durchsichtig, 255 = undurchsichtig. Achtung: Reihenfolge muss beachtet werden.

Environment Mapping: Textur enthält Projektion der Umgebung (für spiegelnde Objekte). Dafür gibt es verschiedene Mapping-Arten wie Sphere Environment Mapping (Kugel, auf die die Umgebung gemappt wird. Sieht nur in kleinem Bereich vom Kugel gut aus) oder Cube Environment Mapping (Speichere pro Objekt sechs Projektionen als Würfel, Zugriff abhängig vom Augenpunkt, an welchem der Lichtstrahl der Spiegelung den Würfel trifft).

Bump Mapping: Durch Modifizierung der Normalenvektoren in Kombination mit Textur können Unebenheiten simuliert werden (z.B. Backsteinmauer, bei Fugen weniger hoch). Implementation: Benötigt Height Mapping (1 Wert, Grauwertmatrix, enthält Höhenänderungen) und Normal Mapping (3 Werte, Formatrix enthält Normalenrichtungsänderung). Achtung: Suggestierte Höhendifferenzen sind von der Seite her nicht sichtbar und haben keine Auswirkung auf Physik.

Displacement Mapping: Textur enthält Angaben zur Veränderung der Geometrie. Vorteile: Displacement Map + grobe Geometrie braucht weniger Platz als feine Geometrie. Eine Geometrie ist mit mehreren Displacements (Skins) nutzbar. Funktionsweise: Jede Fläche des Körpers wird anhand der Displacement Map in mehrere kleine Flächen unterteilt. Die neuen Punkte können nur entlang der Flächennormalen verschoben werden. Sie werden also entweder aus der Fläche herausgehoben oder hineingeschoben. Die Stärke der Verschiebung ist in der Displacement Map als Grauwert hinterlegt. Eignet sich besonders gut, um Landschaften kompakt zu beschreiben.

Netzvereinfachung: Mit Displacement Map ist es auch möglich, die Zahl der Polygone für ein Körper deutlich zu reduzieren und mittels einer Map wiederherzustellen. Ermöglicht einfache Generierung von LOD-Modellen.

17. AUGMENTED REALITY

Unter erweiterter Realität / Augmented Reality versteht man die computergestützte Erweiterung der Wahrnehmung. Kann alle menschlichen Sinne ansprechen, häufig wird jedoch nur die visuelle Darstellung von Informationen verstanden, also die Ergänzung von Bildern und Videos mit computergenerierten Zusatzinformationen oder virtuellen Objekten mittels Einblendung/Überlagerung (XBOX Kinect, HUD).

18. RAYTRACING

Strahlverfolgung eignet sich zur Modellierung von spiegelnden Reflexionen und von Transparenz mit Brechung. Für jeden Pixel wird ausgehend von der Kamera ein Strahl gelegt und der Schnittpunkt mit dem ersten Objekt bestimmt.

Bei spiegelnden Objekten wird der Reflexionsstrahl berechnet und rekursiv weiter behandelt, bei Transparenz wird zusätzlich der gebrochene Strahl weiter behandelt.

Effizienzsteigerung: Berechnung ist sehr aufwändig, durch Z-Buffer, Begrenzungsvolumen und Tree-Erstellung kann die Effizienz etwas gesteigert werden.

19. PHYSIKALISCHE SIMULATIONEN

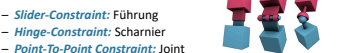
Starre Körper (rigid body): Verformt sich nicht, hat Masse, Trägheitsmoment, momentaner Bewegungszustand und Kollisionen. Einzelne Körper dürfen sich nicht durchdringen. Beim Kontakt werden die Kontaktkräfte berechnet und auf die Körper angewendet.

Physikalische Simulation: Aus dem momentanen Bewegungszustand aller Körper wird mittels Simulation der nächste Zustand abgeleitet. Es gibt zwei Arten der Simulation: Lagrange (mit und ohne reduzierte Koordinaten) und Impulsbasiert.

19.1. CONSTRAINTS
Über Constraints (Joints) können mehrere Körper miteinander verbunden werden. Constraints sind physikalische Zwangsbedingungen (Scharnier einer Türe, Kugellager, ...)

- Slider-Constraint: Führung
- Hinge-Constraint: Scharnier
- Point-to-Point Constraint: Joint

Kraftimpulse müssen so berechnet werden, dass Constraints gültig bleiben.



20. SHADERS
Shader sind Programme zur Beeinflussung der Darstellung von Objekten und Effekten in der Grafikpipeline. Werden in HLSL oder GLSL geschrieben. Sie nutzen parallele Berechnungen für hohe Effizienz und ermöglichen komplexe Effekte wie Licht, Schatten und Spiegelungen direkt auf der GPU.

20.1. ARTEN VON SHADERN
Neben Fragment-Shadern und Vertex-Shadern gibt es auch noch Geometry-Shader (Erzeugen zusätzliche Geometrie) und Tessellation-Shader (Verfeinern Oberflächen).

20.1.1. Fragment-Shader / Pixel Shader
Berechnen Farbe und Eigenschaften einzelner Pixel für realistische Darstellung in 3D-Rendern. Steuern Licht, Farbe und Transparenz von Materialien und Oberflächen. Normierte UV-Koordinaten zwischen 0 und 1 ermöglichen flexible Texturzuordnung auf unterschiedlichen Auflösungen. Fragment-Shader sind zustandslos (da sonst Probleme mit Parallelisierung). Hohe Rechenleistung nötig.

Anwendungsbereiche: Visuelle Effekte, Komplexe Rendering-Techniken (HDR, Bloom, Lensflare), Verzerrungs- und Umwelteffekte (Wasser- & Hitzewellen).

20.1.2. Vertex-Shader
Verändern die Position von Punkten im dreidimensionalen Raum anhand ihrer Koordinaten zur Simulation von Bewegung und Tiefe. Verwenden Funktionen wie Skalierung, Rotation und Verschiebung. Sind die erste Stufe der Rendering-Pipeline und beeinflussen direkt die Objektgeometrie. Sie verarbeiten grosse Datenmengen parallel und sind deshalb für Echtzeit-Grafikanwendungen ideal. Sind zustandslos.

Anwendungsbereiche: Animation von Wasserwellen oder bewegtem Gras, Terrain-Generierung (Höhen von Vertices mit Noise-Texturen)

Besonderheiten: Komplexe Bewegungen müssen in anderen Teilen der Grafik-Engine umgesetzt werden

20.1.3. Gegenüberstellung

Modell	Compute Shader	Vertex Shader	Fragment Shader
Zweck	Allgemeine Berechnungen auf der GPU	Manipulation von 3D-Vertex-Daten	Berechnung der Pixel-Fragmentfarbe
Teil der Pipeline	Nein (unabhängig)	Ja (früher Schritt)	Ja (später Schritt)
Eingabedaten	Beliebige Daten (Buffer, Texturen)	Vertex-Daten (Position, Normale, UV)	Pixel-Fragment-Daten (UV, Textur)
Ausgabe	Beliebige Daten (z.B. Buffer)	Transformierte Vertex-Positionen	Farbwerte (RGBA)
Parallelität	Hohes hoch (Abhängigkeit, Thread)	Niedrig	Niedrig
Typische Anwendungen	Physik, ML, Bildverarbeitung, Noise-Generierung	Animationen, Transform-Verformung	Schattierung, Spiegelungen, Pixel-Processing
Shader-Sprache	HLSL, GLSL, Metal	GLSL, HLSL	GLSL, HLSL